

Algoritmi e Strutture Dati – Parte 2 - 07/02/2019

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

Esercizio B1 – Punti ≥ 8

Scrivere un algoritmo: `int bits(int n)`
che dato un intero n , restituisca il numero di stringhe binarie di n bit che non contengono bit 1 consecutivi.

Discutere informalmente la correttezza della soluzione proposta e calcolare la complessità computazionale.

Ad esempio, per $n = 5$ le stringhe da contare sono le seguenti

```
00000 00001 00010 00100 00101
01000 01001 01010 10000 10001
10010 10100 10101
```

e l'algoritmo deve restituire 13.

Esercizio B2 – Punti ≥ 10

Si consideri un testo T di n caratteri e un pattern P di m caratteri. Abbiamo già visto problemi in cui si chiedeva quante volte il pattern P appare come **sotto-sequenza** di T (ricordate Lucia e i Promessi Sposi?). In questo compito, invece di contare bisogna **elencare** tutti i modi possibili.

Più formalmente, sia $I = [i_1, \dots, i_m]$ una sequenza ordinata contenente m indici di T (ovvero compresi fra 1 ed n).

Diciamo che I genera P da T se e solo se:

$$\forall k, 1 \leq k \leq m : T[i_k] = P[k]$$

Ad esempio, data la stringa $T = \text{"gooddog"}$, la sequenza di indici $I = [4, 6, 7]$ genera il pattern $P = \text{"dog"}$.

Dati un testo T di n caratteri e un pattern P di m caratteri, scrivere un algoritmo:

```
printAll(ITEM[] T, int n, ITEM[] P, int m)
```

che stampa tutte le sequenze di indici di T che generano P .

Discutere informalmente la correttezza della soluzione proposta e calcolare la complessità computazionale.

Ad esempio, si consideri il testo "gooddog" e il pattern "god". L'algoritmo deve stampare (senza commenti):

```
1, 2, 4    # gooddog
1, 2, 5    # gooddog
1, 3, 4    # gooddog
1, 3, 5    # gooddog
```

Cognome:

Nome:

Matricola:

Riga:

Col:

Esercizio B3 – Punti ≥ 12

Si consideri una matrice binaria M di dimensione $n \times n$. Scrivere un algoritmo

```
int cross(int[][] M, int n)
```

che restituisca la dimensione della più grande croce composta da valori 1 presente nella matrice. Una croce è composta da un bit 1 centrale e quattro braccia di uguale lunghezza, contenenti bit 1. Non deve essere necessariamente circondata da bit 0.

Discutere complessità e correttezza dell'algoritmo proposto. Si noti che esistono algoritmi $\Theta(n^2)$ e $\Theta(n^3)$.

Per esempio, la croce visibile nell'esempio è composta da 17 valori 1 e l'algoritmo dovrà restituire 17.

1	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	1
1	1	1	0	1	1	0	1	0	1
0	0	0	0	1	0	0	1	0	0
1	0	0	0	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	0
1	0	0	0	1	0	0	1	0	1
1	0	1	1	1	1	0	0	1	1
1	1	0	0	1	0	0	0	0	1
1	0	1	1	1	1	0	1	0	0